

EasyGLUT2D

EasyGLUT2D – это библиотека для упрощённой работы с FreeGLUT для 2D-графики.

FreeGLUT – это свободное программное обеспечение с открытым исходным кодом, альтернатива библиотеке OpenGL Utility Toolkit (GLUT).

Установка и настройка

Для работы с FreeGLUT (и, соответственно, с EasyGLUT2D) необходим компилятор C и библиотека FreeGLUT.

Linux (Ubuntu, Mint, Debian): `sudo apt update sudo apt install gcc sudo apt install freeglut3-dev`

Windows: Для системы Windows необходимо установить компилятор C (например MinGW) и установить саму библиотеку с официального сайта freeglut: <https://freeglut.sourceforge.net>

Компиляция кода библиотеки

Для C-кода: `gcc main.c EasyGLUT2D.c -o demo -lglut -lGL -lGLU -lm`

Для C++ кода: `g++ main.cpp EasyGLUT2D.c -lglut -lGL -lGLU -lm -o demo`

Где main.c/main.cpp – название компилируемого файла, EasyGLUT2D.c – библиотека для компиляции, demo – название исполняемого файла после компиляции.

Функционал библиотеки

Функции управления окном

Функция `eg2d_window_position(int x, int y)` устанавливает расположение левого верхнего угла создаваемого окна, принимая координаты по x и y в виде целых чисел. Эти значения сохраняются, изменяя глобальные переменные `win_pos_x` и `win_pos_y`. (По умолчанию: 0, 0)

Функция `eg2d_window(int w, int h)` устанавливает размеры создаваемого окна, принимая параметры w (ширина) и h (высота) в пикселях. Эти значения сохраняются, изменяя глобальные переменные `window_width` и `window_height`. (По умолчанию: width = 1920, height = 1080)

Функция `eg2d_axis(float xmin, float xmax, float ymin, float ymax)` является основной функцией, так как она инициализирует библиотеку FreeGLUT, создаёт рабочее окно с границами, которые указаны в аргументе функции, сохраняя их значения. Устанавливает белый фон окна и регистрирует функцию `display()` для отрисовки всех будущих изображений. Автоматически создаёт группу 0. (По умолчанию: xmin=-1.0, xmax=1.0, ymin=-1.0, ymax=1.0)

Функция `eg2d_set_background_color(int r, int g, int b)` устанавливает цвет фона окна, принимая целочисленные значения от 0 до 255 для каждого компонента цвета RGB. (По умолчанию: 255, 255, 255 – белый)

Функция `eg2d_grid_visibility(const char* c)` управляет отображением координатной сетки. Она принимает строковый параметр `c`, который может принимать значения "ON", "On", "on", "1" для включения сетки или "OFF", "Off", "off", "0" для её выключения. При передаче недопустимого значения функция выводит сообщение об ошибке в стандартный поток ошибок. (По умолчанию: включена)

Функция `eg2d_grid_mode_set(const char* m)` задаёт режим отображения сетки. Принимает значения:

- "CM" / "cm" – сантиметровая сетка (шаг 1.0)
- "MM" / "mm" – миллиметровая сетка (шаг 0.1)
- "BOTH" – оба режима одновременно

Функции для работы с группами

Функция `eg2d_create_group(void)` служит для создания новой группы примитивов. Каждому новому примитиву присваивается ID текущей активной группы. Изначально уже существует группа 0 (создаётся автоматически при вызове `eg2d_axis`). Возвращает числовой ID новой группы.

Функция `eg2d_select_group(int gid)` позволяет переключиться на определённую группу: все последующие примитивы будут добавляться в неё.

Функция `eg2d_show_only_group(int gid)` скрывает все группы, затем показывает только указанную. Если `gid < 0` – показывает все группы.

Функция `eg2d_show_groups(int* ids, int count)` показывает сразу несколько групп по массиву ID.

Функция `eg2d_set_group_visibility(int gid, bool visible)` задаёт видимость конкретной группы, не влияя на другие.

Функция `eg2d_hide_all_groups(void)` скрывает все примитивы всех групп.

Функция `eg2d_show_all_groups(void)` показывает все примитивы всех групп.

Функция `eg2d_clear_group(int gid)` удаляет все примитивы указанной группы и саму группу. ID остальных групп сдвигаются вниз.

Функция `eg2d_get_groups_count(void)` возвращает текущее число созданных групп.

Функция `eg2d_cleanup_all(void)` освобождает всю динамическую память, выделенную библиотекой. Рекомендуется вызывать перед завершением программы.

Управление группами с клавиатуры (встроено в библиотеку):

Пробел – циклически переключать отображаемую группу

- A / a – показать все группы
- H / h – скрыть все группы
- G / g – включить/выключить сетку
- M / m – переключить режим сетки (см → мм → оба)
- C / c – удалить текущую активную группу
- Esc – закрыть окно и завершить программу

Функции для работы с точками

Функция `eg2d_set_point_size(float s)` устанавливает размер отображаемых точек, принимая вещественный параметр `s`, который задаёт диаметр точки в пикселях (через `glPointSize`). (По умолчанию: 7.0)

Функция `eg2d_set_point_color(int r, int g, int b)` изменяет цвет точек, принимая целочисленные значения от 0 до 255 для каждого компонента цвета RGB. Внутренне значения делятся на 255 для преобразования в диапазон [0..1], требуемый OpenGL. (По умолчанию: 0, 0, 0 – чёрный)

Функция `eg2d_point(float x, float y)` добавляет новую точку в массив, сохраняя её координаты. Увеличивает счётчик точек и вызывает `glutPostRedisplay()` для перерисовки окна.

Функции для работы с линиями

Функция `eg2d_set_line_width(float w)` устанавливает толщину отображаемых линий в пикселях (через `glLineWidth`). (По умолчанию: 5.0)

Функция `eg2d_set_line_color(int r, int g, int b)` изменяет цвет линий (RGB 0..255). (По умолчанию: 0, 0, 0 – чёрный)

Функция `eg2d_line(float x1, float y1, float x2, float y2)` добавляет новый отрезок, соединяющий точки (x1,y1) и (x2,y2).

Функции для работы с ломаными линиями

Функция `eg2d_set_broken_line_width(float w)` устанавливает толщину ломаных линий. (По умолчанию: 5.0)

Функция `eg2d_set_broken_line_color(int r, int g, int b)` изменяет цвет ломаных линий (RGB 0..255). (По умолчанию: 0, 0, 0 – чёрный)

Функция `eg2d_broken_line(float* xc, float* yc, int vc)` добавляет ломаную линию из `vc` вершин (`vc >= 2`). Массивы `xc[]` и `yc[]` копируются внутри функции.

Функции для работы с векторами/стрелками

Функция `eg2d_set_arrow_width(float w)` устанавливает толщину стержня стрелки. (По умолчанию: 5.0)

Функция `eg2d_set_arrow_color(int r, int g, int b)` изменяет цвет стрелок (RGB 0..255). (По умолчанию: 0, 0, 0 – чёрный)

Функция `eg2d_arrow(float x1, float y1, float x2, float y2)` добавляет стрелку от точки (x1,y1) к точке (x2,y2). Наконечник автоматически рисуется в конечной точке.

Функции для работы с окружностями

Функция `eg2d_set_circle_width(float w)` устанавливает толщину контура окружности. (По умолчанию: 5.0)

Функция `eg2d_set_circle_color(int r, int g, int b)` изменяет цвет контура окружности (RGB 0..255). (По умолчанию: 0, 0, 0 – чёрный)

Функция `eg2d_set_circle_fill(int r, int g, int b, float alpha)` задаёт цвет заливки окружностей. Параметр `alpha` – вещественное число от 0.0 (полностью прозрачный) до 1.0 (непрозрачный). (По умолчанию: синий, `alpha=0.3`)

Функция `eg2d_enable_circle_fill(bool enable)` включает (`true`) или выключает (`false`) заливку для всех окружностей текущей активной группы. Применяется ретроактивно.

Функция `eg2d_circle(float x, float y, float r)` добавляет окружность с центром (x,y) и радиусом `r`.

Функции для работы с эллипсами

Функция `eg2d_set_ellipse_width(float w)` устанавливает толщину контура эллипса. (По умолчанию: 5.0)

Функция `eg2d_set_ellipse_color(int r, int g, int b)` изменяет цвет контура эллипса (RGB 0..255). (По умолчанию: 0, 0, 0 – чёрный)

Функция `eg2d_set_ellipse_fill(int r, int g, int b, float alpha)` задаёт цвет заливки эллипсов. (По умолчанию: синий, `alpha=0.3`)

Функция `eg2d_enable_ellipse_fill(bool enable)` включает/выключает заливку для эллипсов текущей группы.

Функция `eg2d_ellipse(float x, float y, float a, float b)` добавляет эллипс с центром (x,y), полуосью a по оси X и полуосью b по оси Y.

Функции для работы с треугольниками

Функция `eg2d_set_triangle_width(float w)` устанавливает толщину контура треугольника. (По умолчанию: 4.0)

Функция `eg2d_set_triangle_color(int r, int g, int b)` изменяет цвет контура треугольника (RGB 0..255). (По умолчанию: 0, 0, 0 – чёрный)

Функция `eg2d_set_triangle_fill(int r, int g, int b, float alpha)` задаёт цвет заливки треугольников. (По умолчанию: красный, alpha=0.3)

Функция `eg2d_enable_triangle_fill(bool enable)` включает/выключает заливку для треугольников текущей группы.

Функция `eg2d_triangle(float x0, float y0, float x1, float y1, float x2, float y2)` добавляет треугольник по трём вершинам.

Функции для работы с многоугольниками

Функция `eg2d_set_polygon_width(float w)` устанавливает толщину контура многоугольника. (По умолчанию: 4.0)

Функция `eg2d_set_polygon_color(int r, int g, int b)` изменяет цвет контура многоугольника (RGB 0..255). (По умолчанию: 0, 0, 0 – чёрный)

Функция `eg2d_set_polygon_fill(int r, int g, int b, float alpha)` задаёт цвет заливки многоугольников. (По умолчанию: зелёный, alpha=0.3)

Функция `eg2d_enable_polygon_fill(bool enable)` включает/выключает заливку для многоугольников текущей группы.

Функция `eg2d_polygon(float* xc, float* yc, int vc)` добавляет замкнутый многоугольник из `vc` ≥ 3 вершин. Поддерживает как выпуклые, так и вогнутые формы (триангуляция через Ear-Clipping).

Функции для вывода текста

Функция `eg2d_set_outtext_color(int r, int g, int b)` устанавливает цвет текста (RGB 0..255). (По умолчанию: 0, 0, 0 – чёрный)

Функция `eg2d_outtext(float x, float y, const char* text)` выводит строку `text`, позиционируя её левый нижний угол в точке (x,y). Используется битмап-шрифт GLUT.

Функция запуска

Функция `eg2d_show(void)` запускает главный цикл обработки событий FreeGLUT (`glutMainLoop`). В этом цикле происходит:

- обработка событий клавиатуры и закрытия окна,
- последовательная отрисовка: сетка (если включена), координатные оси, все добавленные примитивы.

Примитивы, выходящие за установленные границы (`eg2d_axis`), автоматически отсекаются алгоритмами Лян-Барски (для линий) и Сазерленда-Ходжмана (для полигонов).

ОБНОВЛЕНИЯ В ТЕКУЩЕЙ ВЕРСИИ

- Миллиметровая сетка: добавлена возможность выбора режима сетки через `eg2d_grid_mode_set("CM" / "MM" / "BOTH")`.
- Динамическое переключение: клавиша M переключает режимы сетки во время работы программы, клавиша G включает/выключает отображение сетки.
- Префикс `eg2d_`: все публичные функции теперь имеют префикс `eg2d_` для предотвращения конфликтов имён в больших проектах.
- Улучшенное отсечение: полигоны и заливки корректно обрабатываются при выходе за границы области видимости.
- Прозрачность: заливка фигур поддерживает альфа-канал для создания полупрозрачных эффектов.

Пример использования

```
#include "EasyGLUT2D.h"

void draw_scene(void) {
    const float PI = 3.14159265358979323846f;

    eg2d_select_group(0);

    eg2d_set_point_color(0, 0, 0);
    eg2d_set_point_size(10.0f);
    eg2d_set_line_color(0, 100, 200);
    eg2d_set_line_width(3.0f);
    eg2d_set_arrow_color(200, 50, 50);
    eg2d_set_arrow_width(4.0f);

    eg2d_point(-3.0f, 2.0f);
    eg2d_point(0.0f, 0.0f);
    eg2d_point(3.5f, -2.5f);

    eg2d_line(-5.0f, -5.0f, 5.0f, 5.0f);
    eg2d_line(-4.0f, 3.0f, 4.0f, 3.0f);
    eg2d_line(0.0f, -4.0f, 0.0f, 4.0f);

    {
        float xs[] = {-4.0f, -2.0f, 0.0f, 2.0f, 4.0f};
        float ys[] = {-3.0f, -1.0f, -3.0f, -1.0f, -3.0f};
        eg2d_set_broken_line_color(100, 100, 255);
        eg2d_set_broken_line_width(2.5f);
        eg2d_broken_line(xs, ys, 5);
    }

    eg2d_arrow(-3.0f, 0.0f, 3.0f, 0.0f);
    eg2d_arrow(0.0f, -3.0f, 0.0f, 3.0f);
    eg2d_arrow(-2.0f, -2.0f, 2.0f, 2.0f);

    eg2d_set_outtext_color(0, 0, 0);

    eg2d_create_group();
    eg2d_select_group(1);

    eg2d_set_circle_color(0, 0, 150);
    eg2d_set_circle_width(3.0f);
    eg2d_set_ellipse_color(150, 0, 150);
    eg2d_set_ellipse_width(3.0f);

    eg2d_set_circle_fill(100, 150, 255, 0.4f);
    eg2d_enable_circle_fill(true);
}
```

```

eg2d_set_ellipse_fill(255, 200, 255, 0.3f);
eg2d_enable_ellipse_fill(true);

eg2d_circle(0.0f, 0.0f, 1.5f);
eg2d_circle(-3.0f, 2.0f, 1.2f);
eg2d_circle(4.0f, -3.0f, 1.8f);

eg2d_ellipse(2.0f, 2.0f, 1.0f, 0.5f);
eg2d_ellipse(-2.0f, -2.0f, 0.6f, 1.3f);
eg2d_ellipse(0.0f, -3.5f, 2.0f, 0.8f);

eg2d_enable_circle_fill(false);
eg2d_set_circle_color(255, 100, 0);
eg2d_set_circle_width(4.0f);
eg2d_circle(-4.0f, -4.0f, 0.9f);
eg2d_enable_circle_fill(true);

eg2d_create_group();
eg2d_select_group(2);

eg2d_set_triangle_color(0, 120, 0);
eg2d_set_triangle_width(3.0f);
eg2d_set_triangle_fill(150, 255, 150, 0.5f);
eg2d_enable_triangle_fill(true);

eg2d_triangle(0.0f, 2.0f, -1.5f, -0.5f, 1.5f, -0.5f);
eg2d_triangle(-4.0f, 0.0f, -2.0f, 3.0f, -1.0f, -4.0f);

eg2d_set_polygon_color(120, 0, 120);
eg2d_set_polygon_width(3.0f);
eg2d_set_polygon_fill(220, 180, 255, 0.4f);
eg2d_enable_polygon_fill(true);

{
    float xs[] = {3.0f, 4.0f, 3.5f, 2.5f, 2.0f};
    float ys[] = {0.0f, 1.0f, 2.5f, 2.0f, 0.5f};
    eg2d_polygon(xs, ys, 5);
}

{
    float xs[] = {0.0f, 0.5f, 2.0f, 0.5f, 1.0f, -0.5f};
    float ys[] = {-1.0f, 0.5f, 0.0f, 1.5f, 0.5f, 0.0f};
    eg2d_polygon(xs, ys, 6);
}

{
    float xs[] = {-5.0f, -3.0f, -4.0f, -2.0f, -5.0f};
    float ys[] = {3.0f, 4.5f, 5.5f, 4.0f, 3.0f};
    eg2d_polygon(xs, ys, 5);
}

```



```

}

eg2d_enable_polygon_fill(false);
eg2d_set_polygon_color(255, 50, 50);
eg2d_set_polygon_width(4.0f);
{
    float xs[] = {4.0f, 5.0f, 4.5f, 3.5f};
    float ys[] = {-4.0f, -3.0f, -2.0f, -3.5f};
    eg2d_polygon(xs, ys, 4);
}
eg2d_enable_polygon_fill(true);

eg2d_create_group();
eg2d_select_group(3);

eg2d_set_circle_fill(255, 0, 0, 0.3f);
eg2d_set_ellipse_fill(0, 255, 0, 0.3f);
eg2d_set_triangle_fill(0, 0, 255, 0.3f);

eg2d_enable_circle_fill(true);
eg2d_enable_ellipse_fill(true);
eg2d_enable_triangle_fill(true);

eg2d_circle(-0.8f, 0.0f, 1.2f);
eg2d_ellipse(0.8f, 0.0f, 1.2f, 0.8f);
eg2d_triangle(0.0f, 1.0f, -1.0f, -1.0f, 1.0f, -1.0f);

eg2d_set_outtext_color(255, 0, 0);
eg2d_set_outtext_color(0, 150, 0);
}

int main(void) {
    eg2d_window(1200, 800);
    eg2d_window_position(100, 50);
    eg2d_set_background_color(245, 245, 250);

    eg2d_axis(-5.0f, 5.0f, -4.0f, 4.0f);

    eg2d_grid_visibility("ON");
    eg2d_grid_mode_set("BOTH");

    draw_scene();

    eg2d_show();
    eg2d_cleanup_all();

    return 0;
}

```